

Best Trading Company Software Developer - What to Learn

Overview

This is a broad generalist full-stack posting spanning web (React, JS/TS, HTML/CSS), backend (Node.js, Python, .NET), two databases (MySQL, and implicitly a relational stack), and classic OOP languages (Java, C++). Most of the JD is already covered by your React/Next.js/TypeScript/Node/Python background. The real gaps are Java and .NET — languages/frameworks you haven't used in any project yet.

MUST know before interview

Java (language)

- What: a statically-typed, class-based OOP language, historically the backbone of enterprise backend systems (Spring Boot for web APIs, Android's original language).
- Why: it's an explicit JD requirement even in a JS/TS-heavy stack — likely legacy systems, Android tooling, or a secondary backend service.
- Bridge: your TypeScript already gives you static typing and OOP-adjacent patterns (classes, interfaces). C++ gives you low-level OOP fundamentals (classes, inheritance, memory model). Java sits between the two — familiar OOP syntax, JVM-managed memory instead of manual.
- Learn: basic syntax (classes, interfaces, generics), Maven/Gradle build basics, and if time allows, a Spring Boot "hello world" REST controller — since your REST API experience (Node/Express, tRPC) maps directly onto Spring's controller/service pattern.

.NET (framework, C#)

- What: Microsoft's application framework — C# is the primary language, ASP.NET Core is its web/API framework.
- Why: listed as a required skill; likely an internal tool or legacy Windows-adjacent system alongside the JS stack.
- Bridge: C# syntax is close to TypeScript/Java (typed, class-based, similar control flow). ASP.NET Core's dependency-injection and controller pattern is conceptually similar to how you structure Express/tRPC route handlers.
- Learn: C# basics (types, LINQ syntax), how an ASP.NET Core Web API project is structured (Controllers, Program.cs, appsettings.json), and how EF Core (its ORM) compares to Prisma, which you already know.

GOOD to know

MySQL: you know PostgreSQL and SQL deeply (Skimly's hand-written schema, Vibe's Prisma/Postgres). MySQL syntax and behavior are close enough to Postgres that this is a non-issue in practice — mention the overlap directly if asked.

Working across many languages in one role: the JD implies juggling several stacks. Frame your Clink AI + multi-project experience (React, RN, TypeScript, Python, SQL, Docker) as proof you ramp fast across languages/frameworks when a team needs it.

Not urgent

Given the JD lists ten technologies with no single-domain focus, expect the actual day-to-day work to lean on whichever 2-3 of these the team already uses in production — don't over-invest broadly; confirm which stack is live in the interview and go deep there.